

1. 미분은 "기울기(변화의 빠르기)"를 구하는 것. 예: 엘리베이터

- 5층 → 10층까지 5초 걸렸다.
- 올라간 층수: 5층, 걸린 시간: 5초 1초에 몇 층 올라갔을까?
- 1초에 1층 = 변화량/시간 = 속도 = 변화의 비율

이게 바로 "미분"의 핵심 개념이다. 즉, 미분은 "얼마나 빨리 바뀌는가?"를 계산하는 도구다.

다시 말해, 미분은 변화율(rate of change)을 계산하는 과정이다. 함수의 기울기(접선의 기울기)를 구하는 것~

2. 함수의 미분: 곡선의 기울기를 구하는 것

함수 예: $f(x) = x^2$ ◀ 이걸 그래프로 그리면 곡선이다. 곡선은 직선처럼 일정한 기울기가 없다. 그래서 어느 한 점에서의 기울기(접선의 기울기)를 알고 싶은 것이다. 이걸 구하는 게 미분이다.

예: 함수 $f(x) = x^2$ 에서의 변화율을 살펴보면,

x가 1일 때 기울기(미분값): $f'(1) = 2 \times 1 = 2$

x가 2일 때 기울기(미분값): $f'(2) = 2 \times 2 = 4$

x가 3일 때 기울기(미분값): $f'(3) = 2 \times 3 = 6$

이처럼 x값이 커질수록 기울기도 커지며, 함수는 더 가파르게 증가한다.

기억 : 함수의 미분은 '순간 변화율' = 접선의 기울기를 의미한다.

3. 기울기 구하는 방식 (초 간단)

직선에서 기울기 공식: 기울기=(y의 변화량)/(x의 변화량)

예: $(1, 2) \rightarrow (3, 6)$ 이라면 $\rightarrow$ x는 2만큼, y는 4만큼 변함 $\rightarrow$ 기울기 = $4 / 2 = 2$

그런데 곡선은? 곡선은 한 점에서 기울기를 알아야 하죠. 그래서 아주 가까운 두 점을 찍고, 그 기울기를 계산하는데... 그 거리를 점점 좁혀서 한 점에 가까워질 때의 기울기 이걸 계산한 것이 바로 미분!

보충 설명 : 미분이란 한 점에서의 곡선의 기울기를 알고 싶을 때, 처음에는 그 점 근처의 두 점을 이용한 평균 변화율로 기울기를 대략 측정한다. 이 평균 변화율을 점점 좁혀서 두 점이 사실상 하나의 점으로 가까워질 때의 기울기 - 이 극한값이 바로 미분이다.

수식으로 보면 어떤 함수 $f(x)$ 의 한 점 $x=a$ 에서의 미분값(기울기)은 다음 극한으로 정의된다.

$$f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}$$

여기서 h는 두 점 사이의 거리이고, 이 거리를 점점 작게

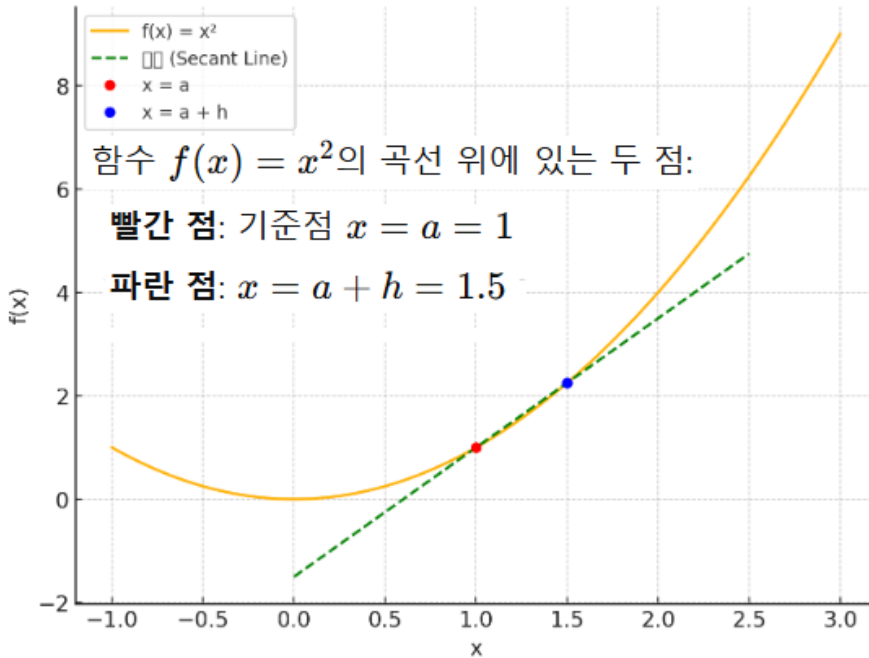
만들어서 (즉, 두 점이 점점 가까워지도록 해서) 실제로 한 점에서의 순간 변화율을 구하는 것이다.

쉽게 말하면

두 점: 평균 기울기 (직선의 기울기),

점점 좁혀서 한 점: 순간 기울기 = 미분

미분의 기하학적 의미: 점점 가까워지는 두 점의 기울기



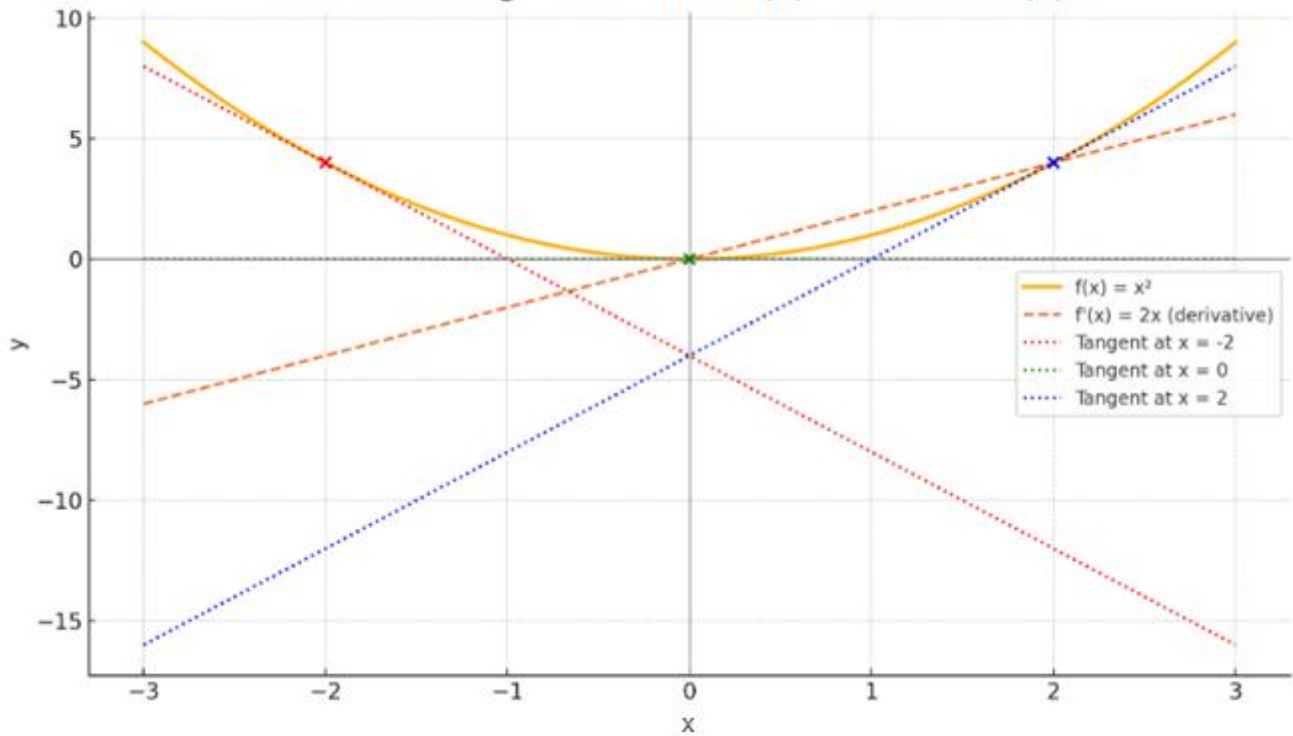
4. 실생활 비유

예시	미분의 의미
자동차	순간 속도 (지금 이 순간 얼마나 빠르게 이동 중인가)
주가	현재 주가가 얼마나 빨리 오르거나 떨어지는가
온도 변화	시간에 따라 온도가 얼마나 급격히 변하는가

5. 다시 정리

- **미분**은 변화량을 계산하는 도구
- 직선 = 항상 일정한 기울기
- 곡선 = 점마다 기울기가 다름 $\rightarrow$ 각 점에서의 **접선 기울기**를 구함
- 이 기울기를 구하는 **행위** = 미분
- 이 기울기를 함수처럼 표현한 것 = **도함수**

Understanding Derivatives: $F(X) = X^2$ And $F'(X) = 2x$



그래프 해석

파란 곡선: $f(x) = x^2$

우리가 미분하고 싶은 원래 함수다. 부드럽게 휘어진 곡선 (포물선)

점선들: 접선 (기울기를 나타냄)

빨강: $x = -2$ 일 때 접선: 기울기 음수(내리막), 초록: $x = 0$ 일 때 접선: 기울기 0(수평선), 파랑: $x = 2$ 일 때 접선 → 기울기 양수(오르막)

- 점선과 연관된 함수: $f'(x) = 2x$

이게 바로 도함수이다. 즉, 각 점에서 얼마나 급하게 변하는지(기울기)를 알려주는 함수다.

한마디로 요약하면? 곡선의 기울기를 계산해서 새로 만든 함수가 도함수고, 그걸 구하는 과정이 미분이다.

미분의 기본 공식 (기울기 정의)

가장 기본적인 미분 정의는 이렇게 생겼어요:
$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

- 이것은 "x에서 아주 조금 떨어진 점과의 평균 기울기를 계산하고, 그 차이를 0에 가깝게 좁히는 것"입니다.

1. 왜 미분이 중요한가? (머신러닝에서)

머신러닝에서 모델은 오류를 줄이는 방향으로 학습한다. 그 방향을 알려주는 나침반 역할을 하는 게 바로 미분이다.

2. 개념 핵심 요약

용어	설명
목적	예측이 틀린 정도(오차)를 줄이는 것
오차 함수 (Loss Function)	정답과 예측의 차이. 이걸 작게 만들고 싶음
미분 (기울기)	오차가 어느 방향으로 줄어드는지 알려줌
경사 하강법(Gradient Descent)	미분 방향을 따라 조금씩 움직이면서 오차를 줄이는 방법

3. 비유로 설명 예: 눈 감고 산 내려가기

- 당신이 산꼭대기(Loss 값이 큼)에 있다고 해보자.
- 눈을 감고 있지만, **발 아래 기울기 방향(=미분)**을 느낄 수 있다.
- 그 방향으로 **조금씩 걸어 내려간다** → 점점 낮은 지점으로!

이렇게 **기울기를 따라 천천히 내려가는 것**이 머신러닝 학습이다.

4. 실제 머신러닝 학습 구조 요약

입력 $x \rightarrow$ 모델 (예: $y = Wx + b$) $\rightarrow$ 예측값 $\hat{y} \rightarrow$ 오차 계산 (Loss) $\rightarrow$ 오차를 W, b 에 대해 미분 $\rightarrow$ 가중치 수정

이 과정을 수천~수만 번 반복하면서 모델이 점점 좋아지는 것이다.

손실 함수 예시 (Loss Function)

예: **Mean Squared Error (평균 제곱 오차)** $L = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$

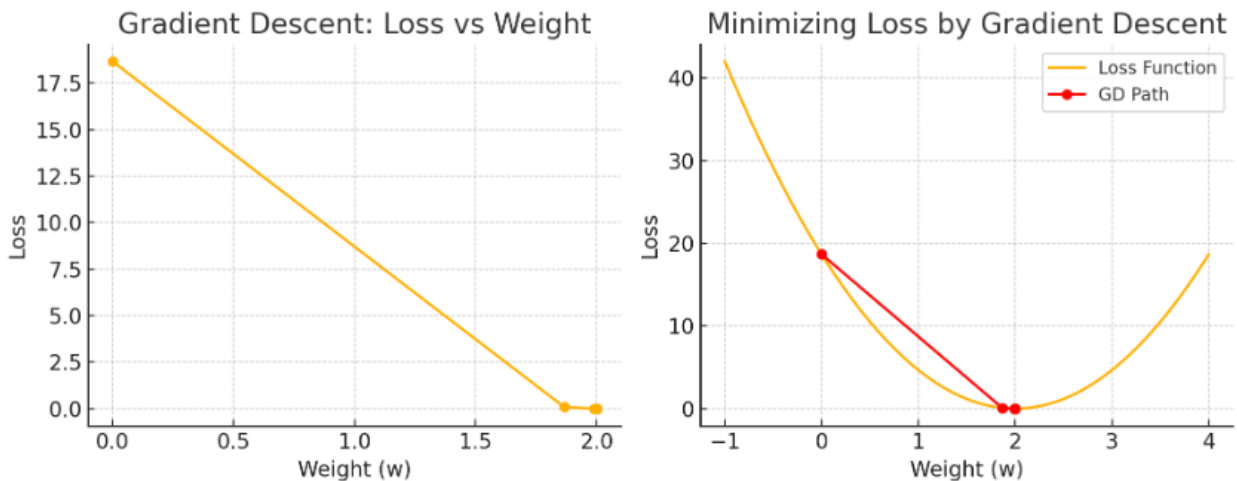
- 여기서 $\hat{y}_i$ 는 모델의 예측값
- 이 Loss를 **미분해서**, 어떤 방향으로 가중치(W)를 바꿔야 오차가 줄어들지 계산

-- 미분이 쓰이는 실제 위치 --

위치	역할	관련 미분
예측 - 정답 비교	얼마나 틀렸는지 계산	Loss 함수 미분
가중치 업데이트	오차가 줄어드는 방향 계산	가중치에 대한 미분 ($\partial L / \partial W$)
활성화 함수	ReLU, Sigmoid 등도 미분이 필요함	Backpropagation 과정에서

7. 핵심 요약 : 미분은 머신러닝에서 "오차를 줄이기 위해 매개변수를 얼마나, 어떤 방향으로 바꿔야 하는가" 를 알려주는 수단이다.

Minimizing Loss By Gradient Descent



이 그래프는 경사하강법이 어떻게 작동하는지를 아주 직관적으로 보여줌

그래프 해설 [왼쪽] 손실 vs 가중치 변화

점점 **Loss(오차)가 줄어드는 모습** : Weight 가 점점 더 좋은 값으로 바뀌면서, Loss 가 내려감
[오른쪽] Loss 곡선 위를 이동하는 경로

파란 곡선은 전체 Loss 함수

빨간 점선은 우리가 ****미분(기울기)****을 따라 움직인 **실제 경로**

머신러닝과의 관계

이 Weight(가중치)는 딥러닝에서는 **신경망의 수많은 연결선**을 의미.

이 수많은 가중치를 동시에 미분해서 **Loss 를 줄이는 방향으로** 조금씩 움직임.

이것을 모든 학습 과정에서 반복하는 게 바로 학습(Training)이다.

* 요약하면 머신러닝은 "오차를 줄이자"는 방안, 미분은 "어디로 가야 오차가 줄지" 알려주는 나침반, 경사하강법은 "그 방향으로 조금씩 이동"하는 알고리즘이다.

딥러닝에서 아주 자주 쓰이는 활성화 함수(Activation Function)의 **미분**을 이해하는 건 매우 중요하다. 왜냐하면 **역전파(Backpropagation)** 때 이들의 미분 값이 학습에 직접 쓰이기 때문이다.

* 1. **활성화 함수란?** 뉴런 하나의 출력 값을 비선형적으로 바꿔주는 함수다.

- **왜 필요할까?**

선형 함수만 있으면 아무리 층을 쌓아도 복잡한 걸 못 배운다. → 비선형 함수로 **학습 능력을 키우기 위해** 활성화 함수가 필요하다.

우리가 볼 대표 2가지

함수 이름	수식	주요 특징
ReLU	$f(x) = \max(0, x)$	간단하고 빠름. 현재 딥러닝 기본값
Sigmoid	$f(x) = \frac{1}{1+e^{-x}}$	0~1 사이. 확률처럼 해석 가능

ReLU 함수와 미분

 **정의:** $f(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$
 **미분:** $f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$

! ReLU는 0 이하일 때 **출력이 0, 미분도 0** → 이걸 "죽은 뉴런(dead neuron)" 문제라고도 해

ReLU 함수는 특별히 **구간별 함수**라서, 미분 과정도 아주 직관적으로 이해할 수 있어요.

 **목표** 우리는 다음 함수를 **직접 미분**해보고 $f(x) = \max(0, x)$

이 결과가 왜 다음처럼 나오는지 이해해볼 겁니다

$$f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x < 0 \end{cases}$$

Step 1: 함수 해석

ReLU 는 다음처럼 두 가지 상황으로 나뉘어요 $f(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$

즉, • x가 양수면 그냥 x (즉, 직선) • x가 0 이하이면 0 (즉, 수평선)

Step 2: 각 구간에서 미분

 ① 구간 1: $x > 0$

$f(x) = x \Rightarrow f'(x) = 1 \rightarrow$ 그냥 기울기 1짜리 직선이므로 미분값도 1입니다.

 ② 구간 2: $x < 0$

$f(x) = 0 \Rightarrow f'(x) = 0 \rightarrow$ y값이 **항상 0인 수평선**이므로 미분값도 0입니다.

 ③ **경계점: $x = 0$** 이걸 미분이 정의되지 않음 😞

왜냐하면, 좌측과 우측에서 기울기가 다르기 때문입니다

- 좌측: 0 • 우측: 1
- $\Rightarrow$ 불연속한 기울기 $\rightarrow x = 0$ 에서는 미분 불가

실제 수학책에서는

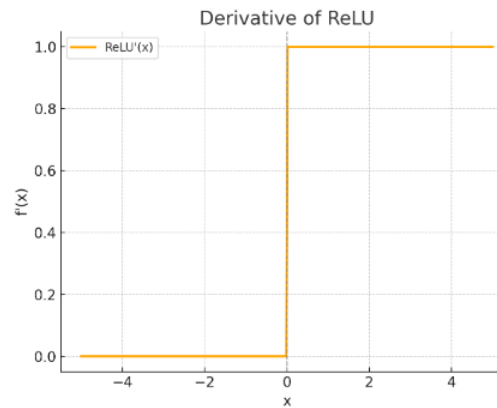
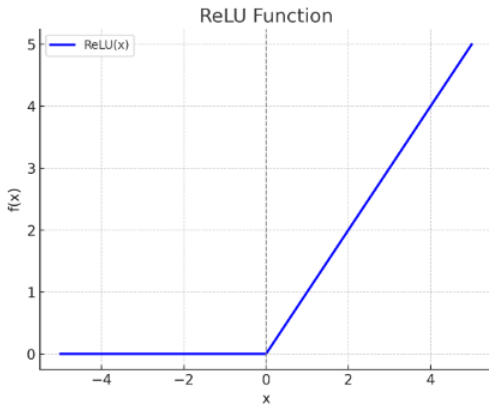
참고 : ReLU는 $x = 0$ 에서 미분 불가능하지만, 신경망 구현에서는 보통 0 또는 1로 임의 지정

결론: 왜 이렇게 되는가?

$$f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x < 0 \\ (\text{보통 } 0) & \text{if } x = 0 \end{cases}$$

이렇게 구간별로 정의된 함수의 경우, **각 구간에서 직접 미분 공식**을 적용해 계산하면 됩니다!

Derivative Of ReLU 이 그래프를 통해 ReLU 함수의 미분 과정을 시각적으로 이해할 수 있어요.



왼쪽: ReLU 함수 $f(x) = \max(0, x)$

- $x > 0$ 일 땐 직선 → 기울기 = 1
- $x \leq 0$ 일 땐 수평선 → 기울기 = 0
- 중간 점선 $x = 0$ 에서 꺾임이 발생

오른쪽: ReLU의 도함수 (미분 결과)

- $x > 0$ 인 구간 → 값이 **1** (기울기 1)
- $x < 0$ 인 구간 → 값이 **0** (기울기 없음)
- $x = 0$ 은 정의되지 않지만 보통 **0** 또는 **1**로 처리

ReLU는 간단하고 효율적인 함수이며, 이처럼 "계단처럼 생긴 도함수" 덕분에 딥러닝에서 학습 효율이 좋습니다. 하지만 $x \leq 0$ 구간에서는 미분값이 0이므로, 해당 뉴런은 학습이 멈추게 될 수 있어요. → "죽은 뉴런" 문제

Sigmoid 함수와 미분

✅ **정의:** $\sigma(x) = \frac{1}{1 + e^{-x}}$ 📊 **미분:** $\sigma'(x) = \sigma(x)(1 - \sigma(x))$

즉, 미분값은 자기 자신의 함수값을 이용해서 구할 수 있어요

예: $\sigma(0) = 0.5$ 이면 $\sigma'(0) = 0.5 \times (1 - 0.5) = 0.25$

요약 정리표

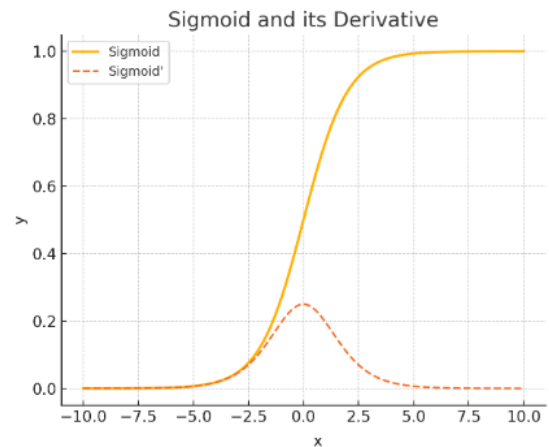
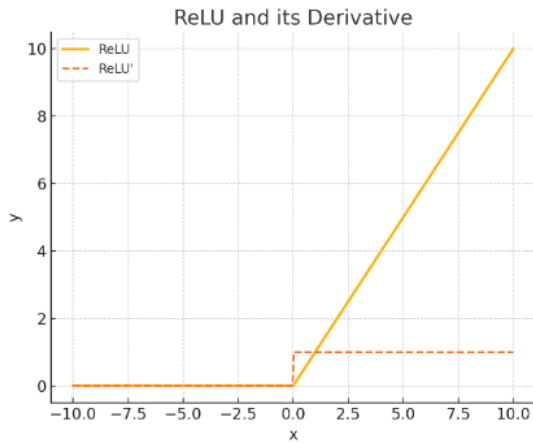
함수	원래 식	도함수 (미분 결과)
ReLU	$\max(0, x)$	1 (if $x > 0$), else 0
Sigmoid	$\frac{1}{1 + e^{-x}}$	$\sigma(x)(1 - \sigma(x))$

실전에서는 어떻게 쓰일까?

신경망 학습할 때 역전파 과정에서 각 뉴런의 출력에 대해 미분값(기울기)을 계산한다.

이때 활성화 함수의 미분 값이 바로 곱해져서 가중치 업데이트에 영향을 준다.

Sigmoid And Its Derivative



그래프를 보면 활성화 함수와 그 미분이 어떻게 생겼는지 명확히 알 수 있어요

왼쪽: ReLU와 그 미분

- 파란 선: ReLU 함수 (0 이하일 땐 0, 그 이후는 직선)
- 주황 점선: 미분값
 - $x > 0$: 기울기 1 (학습 가능)
 - $x \leq 0$: 기울기 0 (학습 불가 = "죽은 뉴런" 위험)

오른쪽: Sigmoid와 그 미분

- 파란 선: Sigmoid 함수 (S자 곡선, 0~1 사이)
- 주황 점선: 미분값
 - 중간($x=0$) 근처에서는 크지만,
 - x 가 너무 크거나 작으면 → 거의 0
→ 기울기 소실 문제 발생

* 요약

함수	장점	단점
ReLU	빠르고 간단, 대부분 사용됨	음수 영역에서는 미분 0 → 죽음
Sigmoid	확률처럼 해석 가능	기울기 소실 문제

1. **역전파란?** 모델이 예측을 하고 나서, 오차를 계산한 후 그 오차를 거꾸로 각 층에 전달하며 가중치들을 조금씩 수정하는 과정이다. 이 과정에서 미분(기울기)이 쓰인다. 왜냐하면 가중치(W)를 얼마나, 어떤 방향으로 바꿔야 오차가 줄어들까? 그걸 알려주는 것이 **미분값**

2. 역전파 기본 구조 : 아래는 딥러닝의 한 뉴런 구조다

$$\text{입력 } x \rightarrow [\text{가중치 } W] \rightarrow z = Wx + b \rightarrow [\text{활성화 함수 } f(z)] \rightarrow \text{출력 } \hat{y}$$

그 후 Loss(예측 $\hat{y}$ 와 정답 y 의 차이) 계산 $\rightarrow$ 이걸 각 층의 W 에 대해 미분해서, 가중치를 조금씩 수정한다.

역전파가 계산하는 것

- 출력층의 오차 $\rightarrow$ 미분으로 역전파
- 곱셈 규칙, 합성 함수 미분(Chain Rule)을 이용해서 각 층의 가중치와 편향에 대해 미분함

예: $\frac{dL}{dW} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{dz} \cdot \frac{dz}{dW}$ • 여기서 $d\hat{y}/dz$ 는 바로 **활성화 함수의 미분**
• 즉, **ReLU** 또는 **Sigmoid**의 도함수가 여기에 직접 들어감

실제로 어떻게 쓰일까?

예: Sigmoid 출력 뉴런

- 예측값: $\hat{y} = \sigma(Wx + b)$
- 오차 (예: MSE): $L = (y - \hat{y})^2$

오차를 W 에 대해 미분하려면? $\frac{dL}{dW} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{dz} \cdot \frac{dz}{dW}$

여기서

- $\frac{dL}{d\hat{y}} \rightarrow$ 오차 함수 미분 • $\frac{d\hat{y}}{dz} \rightarrow$ **Sigmoid**의 도함수 $\sigma(z)(1 - \sigma(z))$
- $\frac{dz}{dW} \rightarrow$ 입력 x

비유로 이해

✖ Loss는 "지금 예측이 얼마나 틀렸는지"

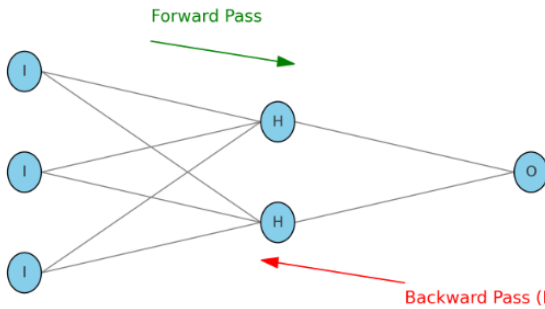
🔵 역전파는 그 오차를 "책임 있는 층에게 되돌려주는 과정"

📐 각 층에서는 "내가 이만큼 틀렸으니, 이만큼만 고쳐야겠다" $\rightarrow$ 이때 **미분이 기준**

* 핵심 요약

개념	역할
Loss	틀린 정도 계산
미분	오차를 줄이는 방향, 속도 계산
활성화 함수 미분	오차를 거꾸로 전달할 때 크기 조절 역할
역전파	오차를 미분값으로 각 층에 전달하고 가중치 수정

Neural Network: Forward And Backward Pass (Backpropagation)



구조 설명

- **Input Layer (좌측):** 입력값 x_1, x_2, x_3
- **Hidden Layer (중앙):** 은닉 뉴런 2개 (비선형 함수 포함)
- **Output Layer (우측):** 예측값 $\hat{y}$

신경망 구조에서 forward pass와 backward pass(역전파)가 어떻게 흐르는지를 직관적으로 보여줍니다.

흐름

✓ 초록색: Forward Pass

- 데이터를 입력 → 각 뉴런을 통과 → 예측값 출력
- 각 뉴런은 $z = w \cdot x + b \rightarrow a = f(z)$
($f(z)$ 는 ReLU나 Sigmoid 같은 활성화 함수)

🔄 빨간색: Backward Pass (역전파)

- 예측이 끝나면, 정답과 비교해 **Loss 계산**
 - 이 Loss를 역으로 각 층에 전달하며
 - 활성화 함수 미분값
 - 오차 미분값
 - 가중치에 대한 미분값
- 이 모든 걸 계산해서 가중치를 업데이트

□ 핵심 요약

- Forward pass = 예측
- Backward pass = 오차 수정 → 이때 **미분이 핵심 역할** (기울기를 계산해서 가중치를 바꾸는 나침반)